

Street Race X

Plataforma de Retos de Carreras Callejeras

Asignatura	Electiva 2 — Desarrollo
Nivel	Avanzado
Modalidad	Proyecto Individual o Grupal (hasta 3 integrantes)
Versión del documento	1.0

1. Contexto del Negocio

Street Race X es una plataforma digital inspirada en la mecánica de aplicaciones de emparejamiento como Tinder, pero aplicada al mundo de las carreras callejeras. La plataforma permite a pilotos y entusiastas de vehículos de alto rendimiento conectarse, retarse mutuamente y competir en diferentes modalidades de carrera, todo dentro de una comunidad estructurada por rangos de habilidad.

La plataforma busca digitalizar y organizar una cultura ya existente en las calles, proporcionando un espacio donde la competencia ocurre entre pilotos del mismo nivel, garantizando equidad y emoción en cada reto. Cada piloto construye su reputación a través de victorias acumuladas, ascendiendo en el sistema de rangos hasta alcanzar la élite.

1.1 Visión del Producto

Ser la plataforma de referencia para la comunidad de pilotos de calle, donde la rivalidad sana, el respeto entre competidores y la emoción de la velocidad convergen en una experiencia digital inmersiva y en tiempo real.

1.2 Propuesta de Valor

- Emparejamiento inteligente entre pilotos del mismo rango y tipo de vehículo.
- Sistema de retos con múltiples modalidades de carrera.
- Perfil detallado del piloto con historial de vehículos y estadísticas.
- Evolución del piloto a través del sistema de rangos.
- Notificaciones y actualizaciones en tiempo real mediante WebSockets.

2. Actores del Sistema

2.1 Usuario / Piloto

Es el actor principal de la plataforma. Un piloto es cualquier persona registrada que posee al menos un vehículo y participa activamente en retos.

- Gestionar su perfil personal e información de vehículos.
- Explorar y descubrir otros pilotos disponibles para retar.
- Enviar y recibir solicitudes de reto.
- Registrar los resultados de las carreras.
- Escalar en el sistema de rangos.

2.2 Administrador del Sistema

Rol con privilegios elevados responsable de la gestión operativa de la plataforma:

- Gestionar usuarios y perfiles.
- Supervisar retos y resolver disputas de resultados.
- Administrar categorías, zonas geográficas y tipos de carrera.
- Acceder a métricas y estadísticas de uso.

3. Entidades del Dominio

3.1 Usuario (User)

id	UUID — Identificador único del usuario
username	String — Nombre de usuario único en la plataforma
email	String — Correo electrónico (único, para autenticación)
password_hash	String — Contraseña encriptada
foto_perfil	String (URL) — Imagen de perfil del piloto
zona_localidad	String — Localidad o barrio de residencia
zona_ciudad	String — Ciudad de residencia
zona_estado	String — Estado / Departamento de residencia
zona_pais	String — País de residencia
rango	Enum (S, A, B, C, D) — Rango competitivo actual
categoria	Enum — Categoría de competencia (ver sección 3.4)
victorias	Integer — Total de retos ganados
derrotas	Integer — Total de retos perdidos
retos_consecutivos	Integer — Contador de victorias consecutivas en el rango actual
estado	Enum (activo, inactivo, suspendido)
created_at	Timestamp — Fecha de registro
updated_at	Timestamp — Última actualización

3.2 Vehículo (Vehicle)

Cada usuario puede registrar hasta 3 vehículos de distintos tipos. Los tipos admitidos son: Automóvil, Motocicleta y Monopatín Eléctrico. El tipo del vehículo activo determina con quién puede competir su propietario.

id	UUID — Identificador único del vehículo
user_id	UUID — FK referencia al propietario
tipo_vehiculo	Enum (auto, moto, monopatin_electrico) — Tipo de vehículo
marca	String — Marca (Toyota, Yamaha, Xiaomi, etc.)
modelo	String — Modelo del vehículo
año	Integer — Año de fabricación

color	String — Color del vehículo
placa	String — Placa / matrícula (única; nullable para monopatines)
foto	String (URL) — Fotografía del vehículo
modificaciones	Text — Descripción de modificaciones técnicas (nullable)
activo	Boolean — Indica si es el vehículo activo para competir
created_at	Timestamp

3.3 Reto (Challenge)

Representa el encuentro competitivo entre dos pilotos. Tiene un ciclo de vida definido desde su creación hasta la declaración de un ganador.

id	UUID — Identificador único del reto
retador_id	UUID — FK usuario que emite el reto
retado_id	UUID — FK usuario que recibe el reto
tipo_carrera	Enum (cuarto_milla, vueltas, derrape)
vehiculo_retador_id	UUID — FK vehículo del retador (define tipo_vehiculo del reto)
vehiculo_retado_id	UUID — FK vehículo del retado para este reto
estado	Enum (pendiente, aceptado, rechazado, en_curso, completado, cancelado)
ganador_id	UUID — FK usuario ganador (null hasta completarse)
ubicacion_acordada	String — Lugar acordado para la carrera
fecha_acordada	Timestamp — Fecha y hora acordada
notas	Text — Observaciones adicionales (nullable)
created_at	Timestamp
updated_at	Timestamp

3.4 Categoría de Competencia (CompetitionCategory)

id	UUID
nombre	String — Nombre de la categoría (Ej: Stock, Modified, Pro)
descripcion	Text — Descripción de los criterios de la categoría
activo	Boolean

3.5 Notificación (Notification)

id	UUID
user_id	UUID — Usuario destinatario
tipo	Enum (reto_recibido, reto_aceptado, reto_rechazado, resultado, rango_subido)
mensaje	String — Contenido de la notificación
leida	Boolean — Estado de lectura
referencia_id	UUID — ID del reto relacionado (nullable)
created_at	Timestamp

4. Sistema de Rangos

El sistema de rangos es el núcleo competitivo de Street Race X. Cada piloto comienza en el rango D y puede ascender hasta el rango S cumpliendo las condiciones establecidas.

Rango	Nombre	Descripción	Condición de Ascenso
S	Leyenda	El rango más alto. Pilotos reconocidos.	No aplica (tope máximo)
A	Élite	Competidores de alto nivel.	Ganar 2 retos consecutivos en rango A
B	Avanzado	Pilotos con experiencia sólida.	Ganar 2 retos consecutivos en rango B
C	Intermedio	Competidores en desarrollo.	Ganar 2 retos consecutivos en rango C
D	Novato	Rango inicial para nuevos usuarios.	Ganar 2 retos consecutivos en rango D

Una derrota resta 1 punto al contador (mínimo 0), sin reiniciarlo por completo ni descender de rango. El rango S no tiene ascenso posterior.

5. Reglas del Negocio

5.1 Registro y Perfil

1. El email y el username son campos únicos; no se permiten duplicados.
2. Un usuario puede registrar como máximo 3 vehículos en su perfil.
3. Solo puede haber un vehículo marcado como 'activo' a la vez por usuario.
4. Un usuario debe tener al menos un vehículo registrado para poder enviar o aceptar retos.
5. Todo usuario nuevo inicia automáticamente en el Rango D.

5.2 Retos

6. Un piloto solo puede retar a otro que tenga exactamente el mismo rango. No se permiten retos entre rangos distintos.
7. El reto solo puede realizarse entre pilotos con el mismo tipo de vehículo activo (auto vs auto, moto vs moto, monopatín vs monopatín).
8. Un piloto no puede retar a otro si ya tiene un reto activo (pendiente, aceptado o en_curso) con esa misma persona.
9. El piloto retado puede aceptar o rechazar el reto. El rechazo no tiene consecuencias para ninguna de las partes.
10. Un reto aceptado debe completarse o cancelarse; no puede quedar en estado indefinido.
11. El resultado final debe ser confirmado por ambas partes o por un administrador en caso de disputa.
12. Un piloto no puede retarse a sí mismo.

5.3 Ascenso de Rango

13. Para ascender, el piloto debe acumular 2 victorias consecutivas en su rango actual.
14. Una derrota resta 1 punto al contador de victorias consecutivas (mínimo 0); no reinicia el contador completo ni desciende el rango.
15. El ascenso es automático al completarse la segunda victoria consecutiva.
16. Al ascender, el contador se reinicia a 0.
17. El rango S es el techo del sistema; no tiene ascenso posterior.
18. El sistema no contempla descenso de rango por derrotas (decisión de diseño v1.0).

5.4 Tipos de Carrera

- Cuarto de Milla (1/4 milla): Aceleración en línea recta. Gana quien cruce primero la línea de meta en 402 metros.
- Carrera por Vueltas: Circuito con número de vueltas acordado entre participantes. Gana quien las complete primero.
- Derrape (Drift): Competencia de habilidad técnica. El ganador se determina por puntuación subjetiva o jueces presenciales.

6. Requerimientos Funcionales

RF-01 — Autenticación y Autorización

- El sistema debe permitir el registro de nuevos usuarios mediante email y contraseña.
- El sistema debe permitir el inicio de sesión y emitir un JWT válido.
- Las rutas protegidas deben validar el token JWT antes de procesar la solicitud.
- El sistema debe implementar roles: piloto y administrador.

RF-02 — Gestión de Perfil

- Los usuarios pueden ver, actualizar y eliminar su propio perfil.
- Los usuarios pueden ver el perfil público de otros pilotos.
- El perfil público muestra: username, foto, rango, categoría, zona, estadísticas y lista de vehículos.

RF-03 — Gestión de Vehículos

- Los usuarios pueden agregar un vehículo (máximo 3), especificando el tipo: Automóvil, Motocicleta o Monopatín Eléctrico.
- Los usuarios pueden editar y eliminar sus vehículos.
- Los usuarios pueden marcar un vehículo como 'activo'.
- El tipo del vehículo activo determina qué pilotos aparecen disponibles para retar.

RF-04 — Gestión de Retos

- Un usuario puede enviar un reto a otro del mismo rango y tipo de vehículo, especificando tipo de carrera y vehículo a utilizar.
- Un usuario puede aceptar o rechazar un reto recibido.
- Un usuario puede cancelar un reto enviado mientras esté en estado 'pendiente'.
- Al completarse un reto, se registra el ganador y se actualizan estadísticas y rango automáticamente.
- Un usuario puede consultar su historial de retos (enviados, recibidos, completados).

RF-05 — Descubrimiento de Pilotos

- El sistema debe listar pilotos del mismo rango y tipo de vehículo activo que el usuario autenticado, con soporte de paginación.
- Debe ser posible filtrar por zona geográfica y/o tipo de carrera preferido.

RF-06 — Notificaciones en Tiempo Real

- El servidor debe emitir eventos en tiempo real cuando un usuario recibe un nuevo reto.
- El servidor debe emitir eventos cuando un reto es aceptado, rechazado o completado.
- El servidor debe emitir un evento cuando un usuario asciende de rango.

7. Requerimientos No Funcionales

- Seguridad: contraseñas hasheadas con bcrypt; JWT con tiempo de expiración.
- Mantenibilidad: código organizado bajo la arquitectura elegida (MVC, Hexagonal o DDD) con nomenclatura TypeScript consistente.
- Usabilidad de la API: respuestas con códigos HTTP apropiados y mensajes de error descriptivos en JSON.
- Documentación: la API debe estar documentada (Swagger/OpenAPI, Postman, Insomnia o similar).
- Persistencia: uso de al menos una base de datos SQL o NoSQL, con justificación de la elección.

8. Stack Tecnológico Requerido

Backend obligatorio

Node.js >= 18

Express.js >= 4

TypeScript >= 5

Socket.io >= 4

JWT (jsonwebtoken)

bcrypt

Base de datos — elegir una

SQL: PostgreSQL o MySQL

NoSQL: MongoDB o Firebase Firestore

Arquitectura — elegir una

MVC (Model-View-Controller)

Hexagonal (Ports & Adapters)

Domain Driven Design (DDD)

9. Definición de Entregas

El proyecto se divide en 3 entregas progresivas. Cada entrega construye sobre la anterior.

Entrega 1 Diseño de la API

Objetivo

Demostrar la capacidad de análisis y diseño de una API RESTful antes de escribir una sola línea de código. Esta entrega es completamente teórica y documental.

Entregables

- Documento de diseño de la API (PDF o Markdown).
- Documentación de endpoints con ejemplos: Postman Collection, Swagger/OpenAPI, Insomnia o herramienta equivalente.
- Diagrama Entidad-Relación (SQL) o diagrama de colecciones (NoSQL), no es obligatorio, sin embargo se desea tener.

Recursos a Cubrir

El estudiante debe identificar y definir todos los recursos necesarios a partir del análisis del contexto del negocio y los requerimientos funcionales. Para cada recurso debe especificar: verbo HTTP, ruta, descripción, parámetros de entrada (path params, query params, body) y estructura de la respuesta esperada.

Como mínimo el diseño debe contemplar recursos para: autenticación, gestión de usuarios/pilotos, vehículos, retos y notificaciones. El estudiante puede identificar y proponer recursos adicionales.

Criterios de Evaluación

- Definición completa de entidades y sus atributos con tipos de datos.
- Diseño del diagrama ER o de colecciones (Se desea tener) .
- Completitud y coherencia de los endpoints (recursos, verbos HTTP, rutas).
- Definición de request/response bodies por endpoint.
- Justificación de la elección de base de datos y arquitectura de software.

Entrega 2 Desarrollo del Backend — API Completa

Objetivo

Implementar la API REST completa en Node.js + Express + TypeScript, aplicando la arquitectura elegida en la Entrega 1, con persistencia en base de datos y autenticación JWT funcional.

Entregables

- Repositorio en GitHub con README completo (instrucciones de instalación, variables de entorno, scripts de ejecución).

- Documentación de la API actualizada con ejemplos de request/response reales (Postman, Swagger, Insomnia o similar).
- Despliegue en la nube (Railway, Render, Heroku, etc.) — opcional, suma puntos adicionales. El mínimo requerido es que la API funcione correctamente en entorno local.

Requisitos de Implementación

- Toda la lógica de negocio de la sección 5 debe estar implementada y validada.
- La estructura de carpetas debe reflejar claramente la arquitectura elegida.
- Variables de entorno (.env) para configuraciones sensibles (DB URI, JWT secret, port).
- Manejo centralizado de errores con respuestas JSON estandarizadas.
- Validación de inputs en los endpoints (Zod, Joi, class-validator o similar).
- Todos los endpoints deben responder con los códigos HTTP correctos.
- Los endpoints protegidos deben validar el JWT correctamente.

Estructura de Respuesta Estándar

Respuesta exitosa	Respuesta de error
<pre>{ "success": true, "data": { ... }, "message": "..."} </pre>	<pre>{ "success": false, "error": "...", "statusCode": 400 } </pre>

Criterios de Evaluación

- Funcionalidad completa de todos los endpoints definidos en la Entrega 1.
- Implementación correcta de todas las reglas del negocio.
- Aplicación coherente de la arquitectura elegida.
- Calidad del código TypeScript: tipado estricto y buenas prácticas.
- README claro y documentación de la API actualizada.

Entrega 3 WebSockets y UI

Objetivo

Integrar WebSockets con Socket.io para comunicación en tiempo real y desarrollar una interfaz de usuario que consuma la API REST y los eventos en tiempo real.

Entregables

- Código del servidor WebSocket integrado en el backend (Socket.io).
- Interfaz de usuario funcional (React, Vue, Angular, o HTML/CSS/JS puro).
- Video demo de máximo 5 minutos mostrando el flujo completo de la aplicación.

Eventos WebSocket a Implementar

Evento	Dirección	Descripción
challenge:received	Server → Cliente	Nuevo reto recibido por el usuario
challenge:accepted	Server → Cliente	El reto fue aceptado por el rival
challenge:rejected	Server → Cliente	El reto fue rechazado por el rival
challenge:completed	Server → Cliente	Resultado de carrera registrado
rank:upgraded	Server → Cliente	El usuario ha ascendido de rango
notification:new	Server → Cliente	Nueva notificación genérica

Vistas Mínimas de la Interfaz

19. Pantalla de Login / Registro.
20. Dashboard: lista de pilotos disponibles para retar (mismo rango y tipo de vehículo).
21. Perfil del piloto: información propia y de otros pilotos.
22. Sección de Retos: listado con acciones (aceptar, rechazar, registrar resultado).
23. Notificaciones en tiempo real: badge o panel que se actualiza vía WebSocket.
24. Gestión de vehículos: agregar, editar y eliminar vehículos del perfil.

Criterios de Evaluación

- Implementación correcta de todos los eventos WebSocket con Socket.io.
- Funcionalidad completa de la UI cubriendo todas las vistas mínimas.
- Integración correcta entre la UI, la API REST y los eventos WebSocket.
- Experiencia de usuario: navegación intuitiva, feedback visual y manejo de estados de carga/error.
- Video demo claro mostrando el flujo completo de la aplicación.

10. Resumen de Entregas

#	Entrega	Enfoque
E1	Diseño de la API	Documentación, entidades y definición de endpoints
E2	Desarrollo del Backend	API REST funcional con arquitectura y reglas de negocio
E3	WebSockets y UI	Comunicación en tiempo real e interfaz de usuario completa

Notas importantes

El plagio o uso de boilerplates sin comprensión demostrable anula la calificación de la entrega correspondiente.

El uso de Inteligencia Artificial como herramienta de apoyo está permitido, siempre que el estudiante pueda explicar y defender cada parte de su implementación.

Si el estudiante no es capaz de defender o explicar la autoría de su proyecto durante la revisión, la nota de la entrega correspondiente será anulada (0).